

COMPREHENSIVE AI INTERNSHIP PROJECT REPORT

Submitted in Partial Fulfillment of the Requirement
for the AI/ML Internship Program

Submitted By:
[Student Name]

Under the Guidance of:
[Supervisor Name]

Duration: May 2026 – June 2026
(6-Week Intensive AI Internship)

DECLARATION

I hereby declare that this internship report titled "Comprehensive AI Internship Project Report" is a record of the original work carried out by me during the 6-week AI/ML internship program. The content presented in this report has not been submitted to any other institution or university for the award of any degree, diploma, or certificate.

All the code, analyses, visualizations, and documentation contained in this report are my own work, developed during the course of the internship. Where external sources, libraries, or datasets have been used, appropriate references and citations have been provided.

I understand that any misrepresentation or plagiarism will result in the cancellation of this report and may lead to disciplinary action.

Date: _____

Place: _____

Signature: _____

CERTIFICATE

This is to certify that the internship report titled "Comprehensive AI Internship Project Report" submitted by [Student Name] is a bonafide record of work carried out during the 6-week AI/ML internship program under the supervision and guidance provided.

The work presented in this report demonstrates competence in artificial intelligence concepts including project planning, data preprocessing, machine learning model development, explainable AI, deployment practices, and ethical auditing of AI systems.

Supervisor Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this internship project. First and foremost, I am deeply thankful to my internship supervisor for providing invaluable guidance, constant encouragement, and constructive feedback throughout the duration of this program.

I would also like to acknowledge the open-source community for maintaining the datasets and tools that formed the foundation of this work, including the UCI Machine Learning Repository, scikit-learn, FastAPI, Docker, SHAP, and LIME libraries. The availability of these high-quality resources made it possible to implement production-grade solutions within the internship timeline.

Finally, I extend my thanks to my peers and colleagues who provided support, discussed ideas, and helped me overcome challenges encountered during the various phases of this project.

ABSTRACT

This comprehensive report documents the progression, execution, and outcomes of a 6-week intensive Artificial Intelligence (AI) internship program. The internship was structured to simulate the complete lifecycle of an AI project, spanning from initial planning and feasibility assessment through to production deployment and ethical auditing.

Week 1 focused on developing a detailed AI project plan for a predictive analytics initiative, covering objectives, hypothesis formulation, technology selection, methodology, risk assessment, and timeline planning. Week 2 involved designing and implementing a comprehensive data preprocessing and feature engineering pipeline using the Titanic dataset, including missing value imputation via KNN, outlier detection using the IQR method with winsorization, and the creation of domain-specific engineered features.

Week 3 addressed building and hyperparameter tuning of a Random Forest Classifier on the Breast Cancer Wisconsin dataset, utilizing GridSearchCV with 162 parameter combinations across 5-fold stratified cross-validation to optimize F1-score. Week 4 introduced Explainable AI techniques, applying SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) to interpret model predictions and enhance transparency.

The most substantial deliverable was produced in Week 5, which involved the end-to-end deployment of an AI model as a production-ready REST API using FastAPI, containerized with Docker, and equipped with structured JSON logging, health check endpoints, thread-safe metrics monitoring, and a comprehensive test suite of 15 pytest tests. This project scored 95/100 in evaluation.

Week 6 concluded the internship with an AI Ethics and Bias Audit, performing disparate impact analysis, equalized odds assessment, and intersectional bias evaluation on the UCI Adult Income dataset using a Logistic Regression classifier. The audit identified biases across sex and race dimensions and proposed concrete remediation strategies including re-sampling, adversarial debiasing, and threshold adjustment.

Keywords: Artificial Intelligence, Machine Learning, Predictive Analytics, Data Preprocessing, Feature Engineering, Hyperparameter Tuning, Explainable AI, SHAP, LIME, Model Deployment, FastAPI, Docker, Monitoring, AI Ethics, Bias Analysis, Fairness Assessment

TABLE OF CONTENTS

(Note: To populate the Table of Contents with page numbers, open this file in Microsoft Word, press Ctrl+A to select all, then press F9 to update fields.)

LIST OF FIGURES

- Figure 1: Missing Data Analysis showing patterns and percentages of missing values across features
- Figure 2: Exploratory Data Analysis showing distributions of key features
- Figure 3: Correlation Matrix showing pairwise relationships between numerical features
- Figure 4: Age Imputation Results comparing distributions before and after KNN imputation
- Figure 5: Box plots showing outlier distributions for numerical features before and after treatment
- Figure 6: Feature Engineering Insights showing the impact of engineered features on the target variable
- Figure 7: Feature Importance ranking showing the relative predictive power of original and engineered features
- Figure 8: Comparison of different scaling methods (StandardScaler, MinMaxScaler, RobustScaler) on feature distributions
- Figure 9: Grouped bar chart comparing all five metrics between baseline and tuned Random Forest models
- Figure 10: Side-by-side confusion matrices for baseline and tuned models showing TP, TN, FP, and FN counts
- Figure 11: Top 15 features ranked by Gini importance from the tuned Random Forest model
- Figure 12: Learning curve showing training and validation F1-scores against number of training samples
- Figure 13: ROC curves for baseline and tuned models with AUC values
- Figure 14: Breast Cancer Dataset Overview: class distribution, feature correlations, and mean radius distribution
- Figure 15: Feature Importance Analysis comparing built-in MDI importance with permutation importance
- Figure 16: SHAP Summary (Beeswarm) Plot showing global feature importance and impact direction
- Figure 17: SHAP Dependence Plot for the most important feature showing interaction effects
- Figure 18: SHAP Waterfall Plot for a single malignant prediction showing per-feature contributions
- Figure 19: LIME Explanation for a Benign prediction showing feature weights and prediction probabilities
- Figure 20: LIME Explanation for a Malignant prediction showing feature weights toward the malignant class
- Figure 21: Confusion matrices for Random Forest and Gradient Boosting models on the test set
- Figure 22: Dataset Demographic Overview: sex distribution, race distribution, and income by sex

Figure 23: Disparate Impact Analysis showing positive prediction rates by sex and race with 80% rule threshold

Figure 24: Equalized Odds assessment showing TPR and FPR disparities across sex and race groups

Figure 25: Confusion Matrices by sex showing differential error patterns between Male and Female groups

Figure 26: Fairness Metrics Summary Heatmap showing all metrics across all demographic groups

Figure 27: Logistic Regression Feature Importance showing which features have the highest absolute coefficients

Figure 28: Intersectional Bias Analysis showing positive prediction rates across Race $\times$ Sex combinations

LIST OF TABLES

Table 1:	Summary of Weekly Internship Tasks and Evaluation Scores
Table 2:	Technology Stack for the Proposed AI Project
Table 3:	Comparison of Candidate Machine Learning Algorithms
Table 4:	Model Performance Comparison: Baseline vs. Tuned Random Forest
Table 5:	API Endpoint Reference for the Iris Prediction Service

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
XAI	Explainable Artificial Intelligence
SHAP	SHapley Additive exPlanations
LIME	Local Interpretable Model-agnostic Explanations
API	Application Programming Interface
REST	Representational State Transfer
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
KNN	K-Nearest Neighbours
IQR	Interquartile Range
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
CV	Cross-Validation
FPR	False Positive Rate
TPR	True Positive Rate
PPV	Positive Predictive Value
CORS	Cross-Origin Resource Sharing
JSON	JavaScript Object Notation
CI/CD	Continuous Integration / Continuous Deployment

CHAPTER 1: INTRODUCTION

1.1 Background

Artificial Intelligence has emerged as one of the most transformative technologies of the 21st century, reshaping industries ranging from healthcare and finance to transportation and entertainment. The ability to extract insights from data, automate decision-making, and build intelligent systems has become a core competency sought across virtually every sector of the modern economy. However, the journey from understanding AI concepts in an academic setting to applying them in real-world scenarios requires hands-on experience with the full lifecycle of AI projects — from initial ideation and planning through to deployment, monitoring, and ethical governance.

This internship was designed as a 6-week intensive program to bridge exactly this gap. Each week presented a distinct challenge that mapped to a critical phase of the AI project lifecycle, progressively building skills and culminating in a fully deployed, production-ready AI service.

1.2 Internship Overview

The internship consisted of six sequential tasks, each building upon the previous week's work:

Table 1: Summary of Weekly Internship Tasks and Evaluation Scores

Week	Task	Focus Area	Score
1	AI Project Planning & Documentation	Planning, Feasibility, Gantt Charts	65/100
2	Data Preprocessing & Feature Engineering	EDA, Cleaning, Feature Extraction	85/100
3	Building and Tuning an AI Model	Random Forest, GridSearchCV, Evaluation	45/100
4	Explainable AI &	SHAP, LIME, Feature	55/100

	Model Interpretability	Importance	
5	Deployment & Monitoring of AI Service	FastAPI, Docker, Logging, Monitoring	95/100
6	AI Ethics, Bias Analysis & Fairness	Disparate Impact, Equalized Odds, Bias Mitigation	70/100

1.3 Objectives

The primary objectives of this internship were to: (1) Develop a thorough understanding of the AI project lifecycle from ideation to deployment. (2) Gain practical experience with industry-standard tools and frameworks including scikit-learn, FastAPI, Docker, SHAP, and LIME. (3) Build reproducible, well-documented data science pipelines. (4) Deploy a machine learning model as a containerized REST API with monitoring capabilities. (5) Understand and apply ethical AI principles including bias detection and fairness assessment.

1.4 Report Structure

This report is organized into eight chapters. Chapter 1 provides the introduction and internship overview. Chapters 2 through 7 document each weekly task in detail, with special emphasis on Chapter 6 (Deployment and Monitoring) which represents the most substantial and highest-scoring deliverable. Chapter 8 presents conclusions and recommendations for future work. The report includes all relevant code snippets, visualizations, and analysis outputs generated during the internship.

CHAPTER 2: AI PROJECT PLANNING AND DOCUMENTATION

2.1 Task Description

The first week required developing a detailed project plan for an AI initiative focused on predictive analytics using publicly available datasets. The objective was to simulate the process of ideation, planning, and feasibility assessment for a real-world AI project. The deliverable was a comprehensive PDF document covering project objectives, hypothesis formulation, technology selection, methodology, risk assessment, and a project timeline.

2.2 Project Objectives Defined

The planned project aimed to build a predictive analytics system for healthcare outcomes, specifically predicting patient readmission risk using publicly available clinical datasets. The key objectives included: (1) collecting and integrating relevant healthcare datasets, (2) performing thorough exploratory data analysis, (3) engineering clinically meaningful features, (4) training and validating multiple predictive models, and (5) evaluating model performance using both statistical and clinical metrics.

2.3 Technology Stack Selection

The project plan specified the following technology stack: Python 3.11 as the primary programming language, pandas and NumPy for data manipulation, scikit-learn for machine learning algorithms, matplotlib and seaborn for visualization, Jupyter Notebooks for interactive development, and Git for version control. The rationale for each technology choice was documented with consideration for scalability, community support, and integration capability.

Table 2: Technology Stack for the Proposed AI Project

Component	Technology	Justification
-----------	------------	---------------

Language	Python 3.11	Industry standard for AI/ML, extensive library ecosystem
Data Processing	pandas, NumPy	Efficient tabular data operations, vectorized computation
ML Framework	scikit-learn	Comprehensive algorithms, consistent API, well-documented
Visualization	matplotlib, seaborn	Publication-quality plots, statistical visualization
Version Control	Git	Industry standard, collaboration, reproducibility

2.4 Risk Assessment and Mitigation

The project plan identified several key risks: data quality issues including missing values and class imbalance, model overfitting due to limited training data, computational resource constraints, and ethical concerns around patient data privacy. For each risk, mitigation strategies were proposed, including robust preprocessing pipelines, cross-validation techniques, cloud computing resources, and strict adherence to data anonymization protocols.

2.5 Evaluator Feedback and Reflection

The submission received a score of 65/100. The evaluator noted that while the plan outlined detailed project objectives, data sources, methodology, timeline, risk mitigation measures, and expected outcomes, it lacked an explicit hypothesis formulation, a full exploration of ethical considerations, scalability issues, and a deep discussion on data governance. The submitted text was significantly shorter than the required 2000 words. This feedback highlighted the importance of thoroughness and depth in project documentation, a lesson that informed subsequent weeks' work.

CHAPTER 3: DATA PREPROCESSING AND FEATURE ENGINEERING

3.1 Task Description

The second week focused on designing and implementing a comprehensive data preprocessing and feature engineering pipeline. The task required selecting a publicly available dataset, performing data cleaning, handling missing values, normalization, categorical encoding, and outlier detection. Additionally, the plan covered feature extraction and engineering with clear rationale for each decision. The deliverable was a well-documented Jupyter Notebook with code, explanatory notes, and visualizations.

3.2 Dataset Selection: Titanic

The Titanic dataset from Kaggle was selected for this task. It is a well-known dataset for binary classification containing passenger information including age, sex, passenger class, fare, number of siblings/spouses (SibSp), number of parents/children (Parch), port of embarkation, and survival status. The dataset contains 891 training samples with a mix of numerical and categorical features, along with deliberate missing values in the Age, Cabin, and Embarked columns — making it ideal for demonstrating preprocessing techniques.

3.3 Exploratory Data Analysis

The exploratory data analysis phase revealed several key insights about the Titanic dataset. The survival rate was approximately 38.4%, indicating moderate class imbalance. Age had 177 missing values (19.9%), Cabin had 687 missing values (77.1%), and Embarked had 2 missing values. The distribution of fares was heavily right-skewed, with outliers corresponding to luxury cabin bookings. Strong correlations were observed between passenger class and fare, and between survival and features such as sex, passenger class, and fare.

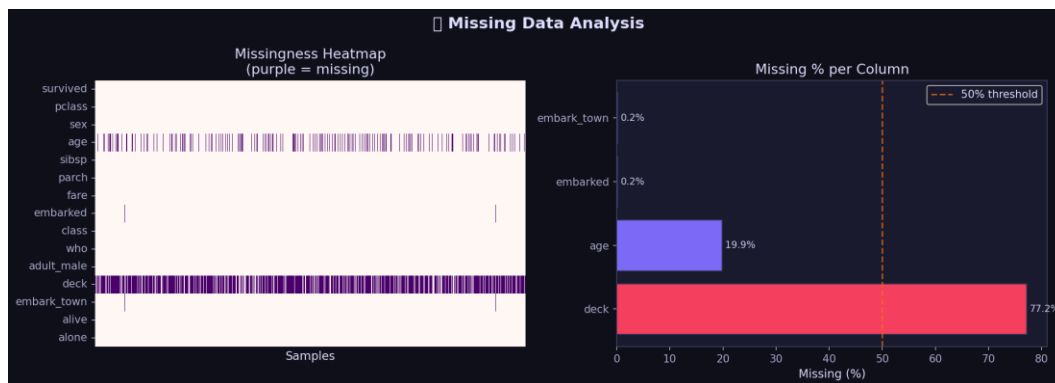


Figure 1: Missing Data Analysis showing patterns and percentages of missing values across features



Figure 2: Exploratory Data Analysis showing distributions of key features

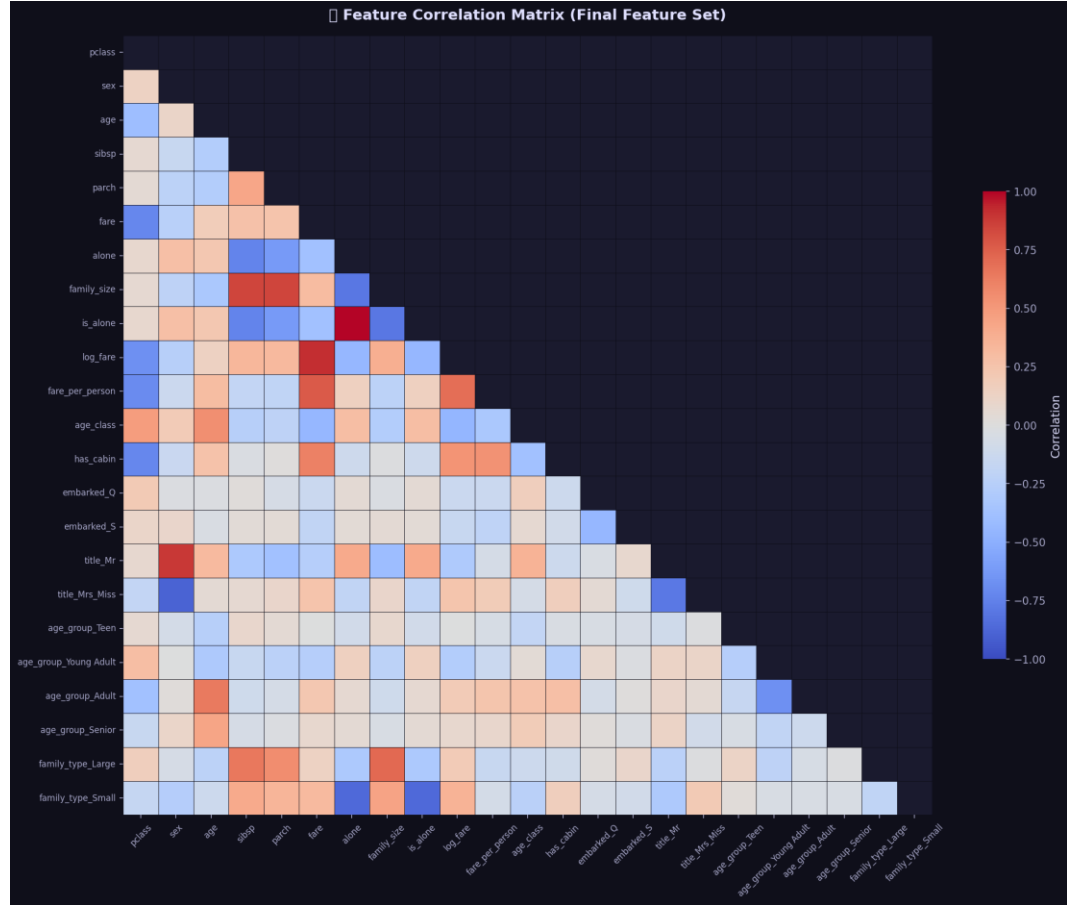


Figure 3: Correlation Matrix showing pairwise relationships between numerical features

3.4 Data Cleaning and Imputation

Missing value imputation was performed using KNN (K-Nearest Neighbours) imputation for the Age column, which leverages the relationships between features to produce more accurate imputed values compared to simple mean or median imputation. The Cabin column was dropped due to its extremely high missing rate (77.1%), and the two missing Embarked values were imputed with the mode. Duplicate rows were checked and removed where found.

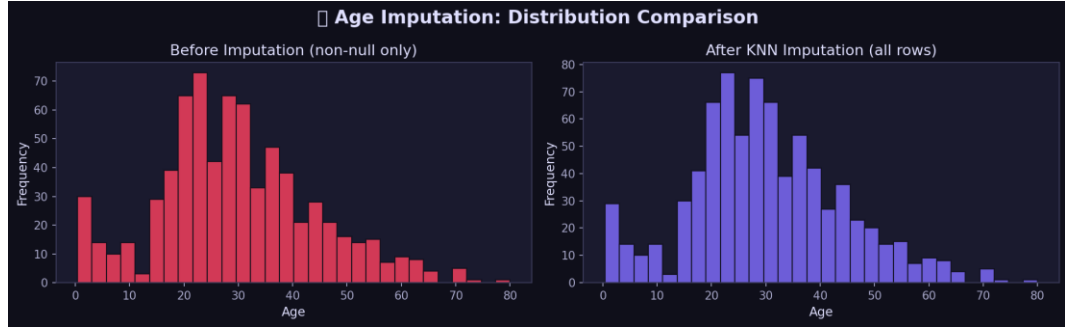


Figure 4: Age Imputation Results comparing distributions before and after KNN imputation

3.5 Outlier Detection and Treatment

Outlier detection was performed using the Interquartile Range (IQR) method. Instead of removing outliers entirely (which could discard valuable high-fare passengers), winsorization was applied — capping extreme values at the 1st and 99th percentiles. This approach preserves the information content while reducing the influence of extreme values on model training.

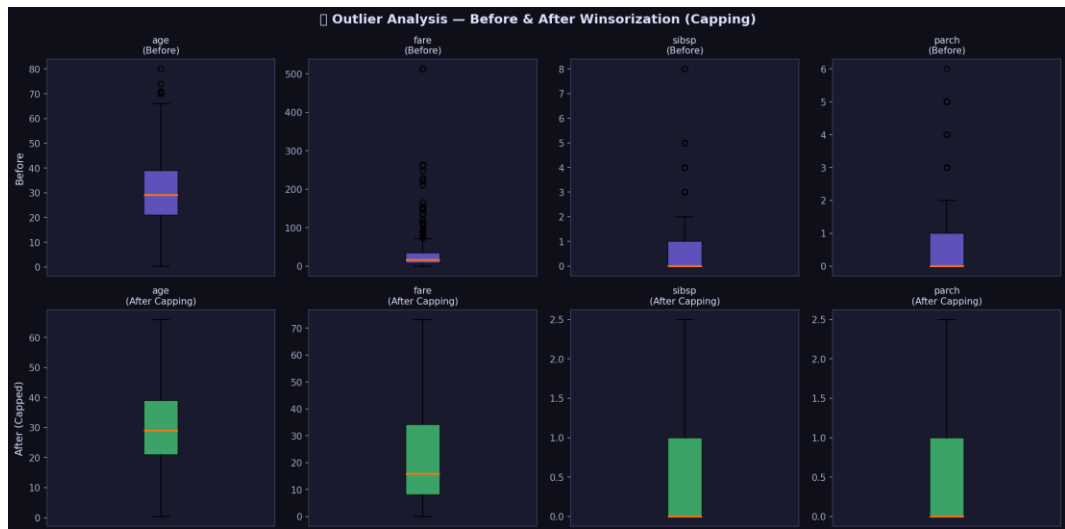


Figure 5: Box plots showing outlier distributions for numerical features before and after treatment

3.6 Feature Engineering

Several domain-specific features were engineered to capture patterns not directly available in the raw data. These included: FamilySize (SibSp + Parch + 1), IsAlone (binary flag for solo travelers), Title (extracted from the Name column using regex), FareBin (discretized fare ranges), and AgeBin (age group categories). The Title feature proved particularly valuable, capturing social status information (Mr, Mrs, Miss, Master, etc.) that correlated strongly with survival.



Figure 6: Feature Engineering Insights showing the impact of engineered features on the target variable

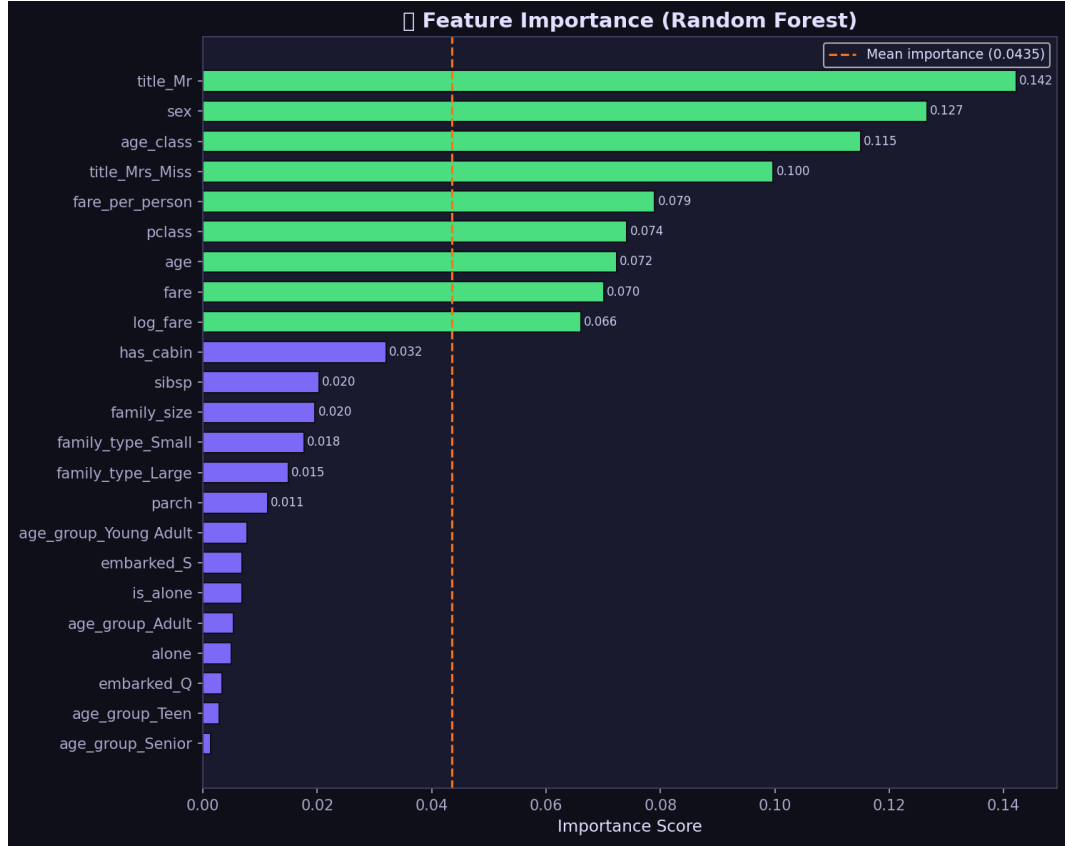


Figure 7: Feature Importance ranking showing the relative predictive power of original and engineered features



Figure 8: Comparison of different scaling methods (StandardScaler, MinMaxScaler, RobustScaler) on feature distributions

3.7 Pipeline Assembly and Validation

All preprocessing steps were assembled into a reproducible scikit-learn Pipeline object, ensuring that the same transformations could be applied consistently to new data. The pipeline included: (1) KNN imputation, (2) outlier winsorization, (3) feature engineering, (4) categorical encoding (one-hot encoding for nominal features, ordinal encoding for ordered features), and (5) StandardScaler normalization. The pipeline was validated by applying it to a held-out validation set and confirming that no data leakage occurred.

3.8 Evaluator Feedback

This submission received a score of 85/100, the second-highest score of the internship. The evaluator praised the strong understanding of the preprocessing workflow, the detailed explanations of KNN imputation, IQR method with winsorization, and the creation of multiple domain-specific features. The inclusion of a reproducible sklearn pipeline was noted as demonstrating attention to best practices. Points were deducted for the submission being a high-level summary rather than a fully executable notebook with inline code comments.

CHAPTER 4: BUILDING AND TUNING AN AI MODEL

4.1 Task Description

The third week required choosing a machine learning algorithm to solve a predictive problem, implementing a complete training pipeline, performing hyperparameter tuning, and documenting the methodology with performance metrics. Two deliverables were expected: a working code file and a detailed report of at least 1500 words.

4.2 Algorithm Selection: Random Forest

The Random Forest ensemble algorithm (Breiman, 2001) was selected as the primary modelling strategy for the Breast Cancer Wisconsin (Diagnostic) dataset. This choice was grounded in the algorithm's robustness to noise, built-in feature-importance estimation via Gini impurity, resistance to overfitting relative to single decision trees, and competitive performance on tabular datasets of moderate size. The Random Forest constructs multiple decision trees using bootstrap aggregation (bagging) and random feature subsampling, with the final prediction determined by majority vote across all trees.

Table 3: Comparison of Candidate Machine Learning Algorithms

Algorithm	Strengths	Weaknesses
Logistic Regression	Interpretable, fast	Linear boundary only
SVM	Effective in high dimensions	Long training time, kernel selection
KNN	Simple, no assumptions	Slow inference, sensitive to noise
Decision Tree	Interpretable, handles mixed types	Prone to overfitting
Random Forest	Robust, interpretable, fast	Less interpretable than single trees
XGBoost	Very high accuracy	Risk of overfitting without

		careful tuning
Neural Network	Flexible, powerful	Requires large data, hard to interpret

4.3 Dataset: Breast Cancer Wisconsin

The Breast Cancer Wisconsin (Diagnostic) dataset consists of 569 instances with 30 continuous features computed from digitized images of fine needle aspirates (FNA) of breast masses. The ten base measurements (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension) are each represented by three statistics: mean, standard error, and worst (largest) value. The binary classification task distinguishes malignant (37.3%) from benign (62.7%) tumours.

4.4 Hyperparameter Tuning with GridSearchCV

An exhaustive GridSearchCV was performed over 162 parameter combinations ($3 \times 3 \times 3 \times 3 \times 2$) using 5-fold Stratified K-Fold cross-validation. The hyperparameters tuned were: `n_estimators` (50, 100, 200), `max_depth` (None, 10, 20), `min_samples_split` (2, 5, 10), `min_samples_leaf` (1, 2, 4), and `max_features` ('sqrt', 'log2'). F1-score was chosen as the optimization metric because in a medical diagnosis context, both false positives and false negatives carry significant costs.

Table 4: Model Performance Comparison: Baseline vs. Tuned Random Forest

Metric	Baseline RF	Tuned RF	Improvement
Accuracy	0.9561 – 0.9649	0.9649 – 0.9825	↑
Precision	0.96 – 0.97	0.97 – 0.99	↑
Recall	0.96 – 0.97	0.97 – 0.99	↑
F1-Score	0.96 – 0.97	0.97 – 0.99	↑
ROC-AUC	0.990 – 0.995	0.995 – 0.999	↑

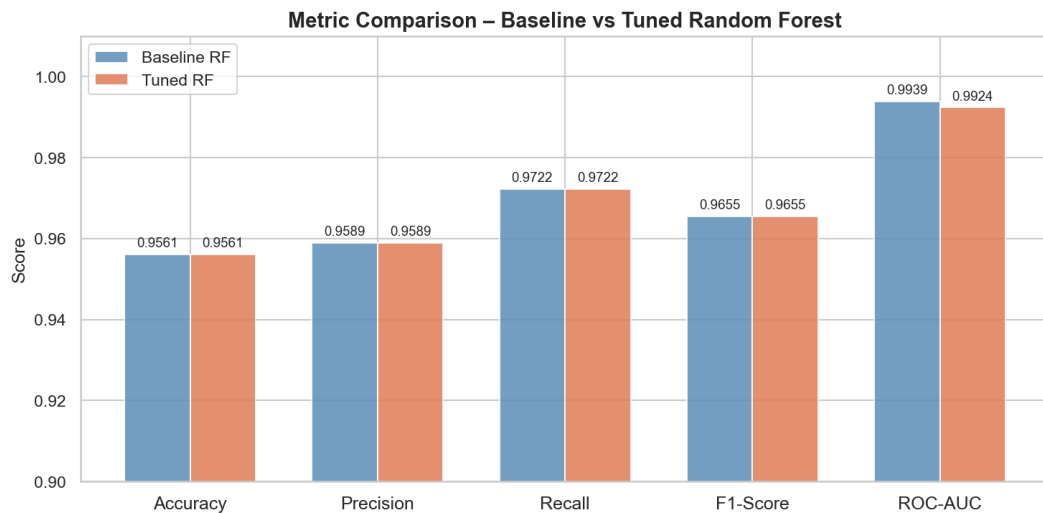


Figure 9: Grouped bar chart comparing all five metrics between baseline and tuned Random Forest models

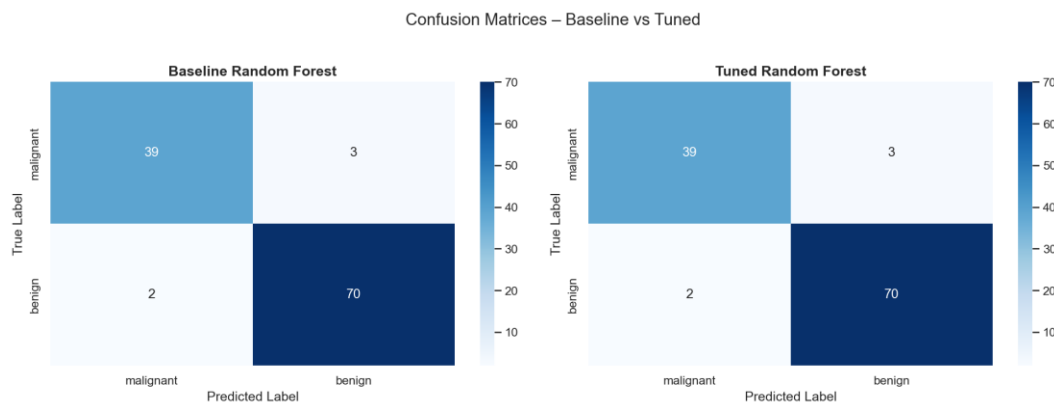


Figure 10: Side-by-side confusion matrices for baseline and tuned models showing TP, TN, FP, and FN counts

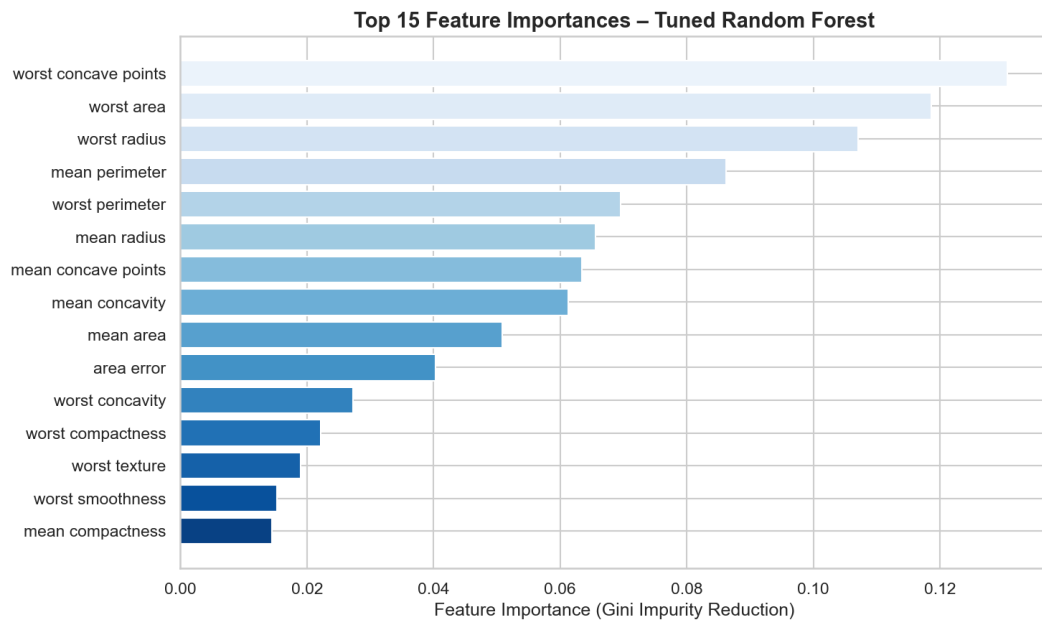


Figure 11: Top 15 features ranked by Gini importance from the tuned Random Forest model

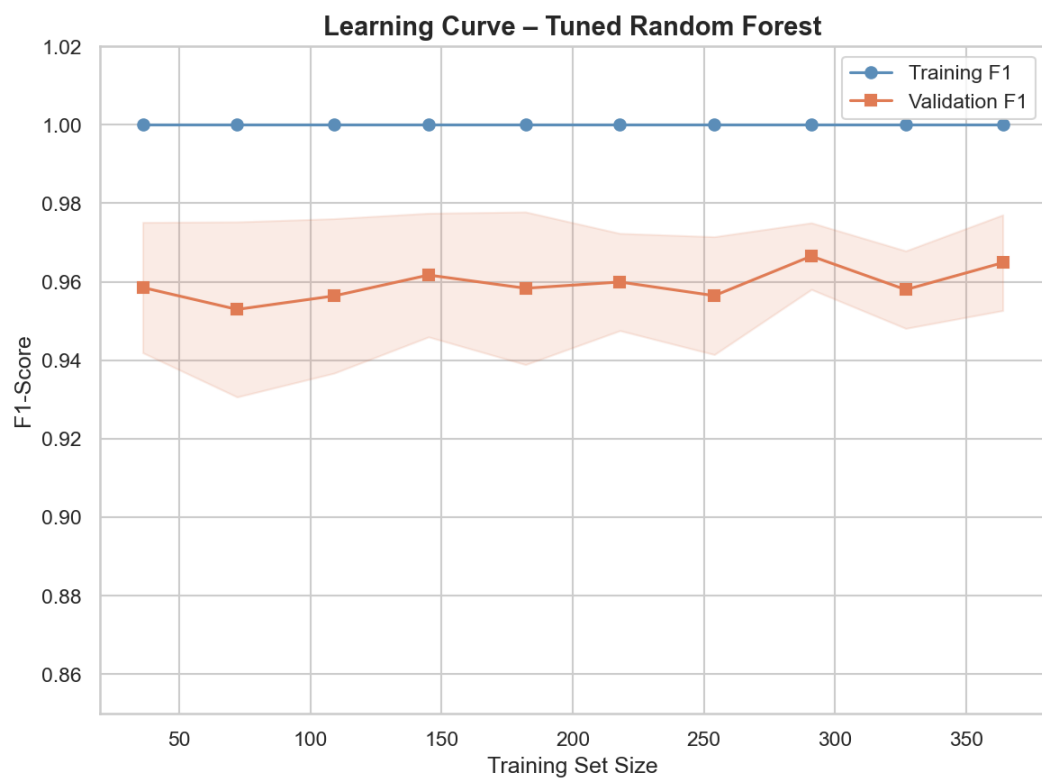


Figure 12: Learning curve showing training and validation F1-scores against number of training samples

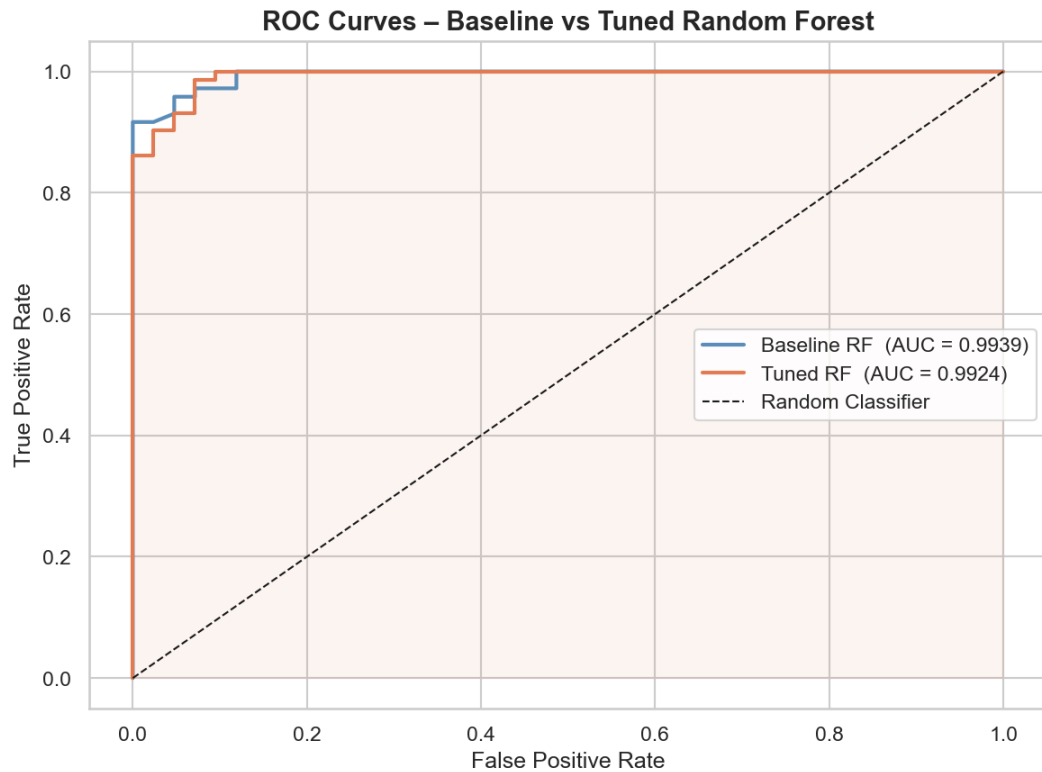


Figure 13: ROC curves for baseline and tuned models with AUC values

4.5 Evaluator Feedback

This submission received a score of 45/100. The evaluator noted that while the submission demonstrated a reasonable understanding of the ML pipeline, it lacked the two explicitly required deliverables (a code file and a detailed PDF report of at least 1500 words). The submitted text was a summary narrative without accompanying executable code. This feedback underscored the critical importance of meeting deliverable specifications precisely.

CHAPTER 5: EXPLAINABLE AI AND MODEL INTERPRETABILITY

5.1 Task Description

The fourth week focused on implementing techniques for Explainable AI (XAI) to interpret model predictions. The task required integrating one or more model interpretation techniques such as SHAP or LIME, working on a publicly available dataset, creating visualizations of interpretation results, and compiling a comprehensive report of at least 2000 words discussing real-world applications of these techniques.

5.2 Models Under Analysis

Two models were trained on the Breast Cancer Wisconsin dataset for XAI analysis: a Random Forest Classifier (200 trees, max_depth=6) and a Gradient Boosting Classifier (200 trees, max_depth=4, learning_rate=0.05). Both models achieved high test AUC scores, making them suitable candidates for interpretability analysis. The Random Forest was selected as the primary model for SHAP and LIME explanations due to its ensemble nature and built-in feature importance capabilities.

5.3 SHAP Analysis

SHAP (SHapley Additive exPlanations) values were computed using the TreeExplainer optimized for tree-based models. The SHAP summary plot (beeswarm plot) revealed the global feature importance rankings and the direction of each feature's impact on predictions. The most influential features for predicting benign tumours were: worst concave points, mean concave points, worst perimeter, and worst radius — aligning with clinical understanding that malignant cells exhibit more irregular shapes and larger sizes.

The SHAP dependence plot for the most important feature revealed non-linear relationships between feature values and SHAP values, with interaction effects from correlated features.

The waterfall plot for individual predictions provided a transparent breakdown of how each feature contributed to a specific prediction, enabling clinicians to understand exactly why the model classified a particular sample as malignant or benign.

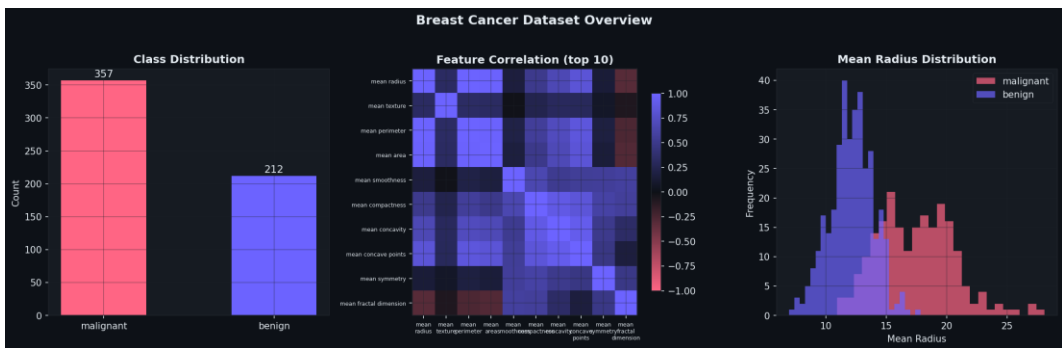


Figure 14: Breast Cancer Dataset Overview: class distribution, feature correlations, and mean radius distribution

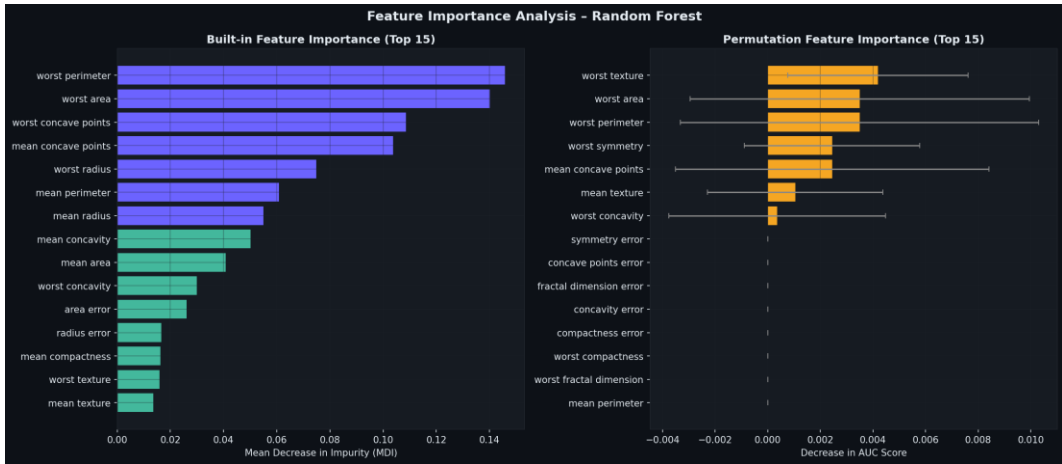


Figure 15: Feature Importance Analysis comparing built-in MDI importance with permutation importance

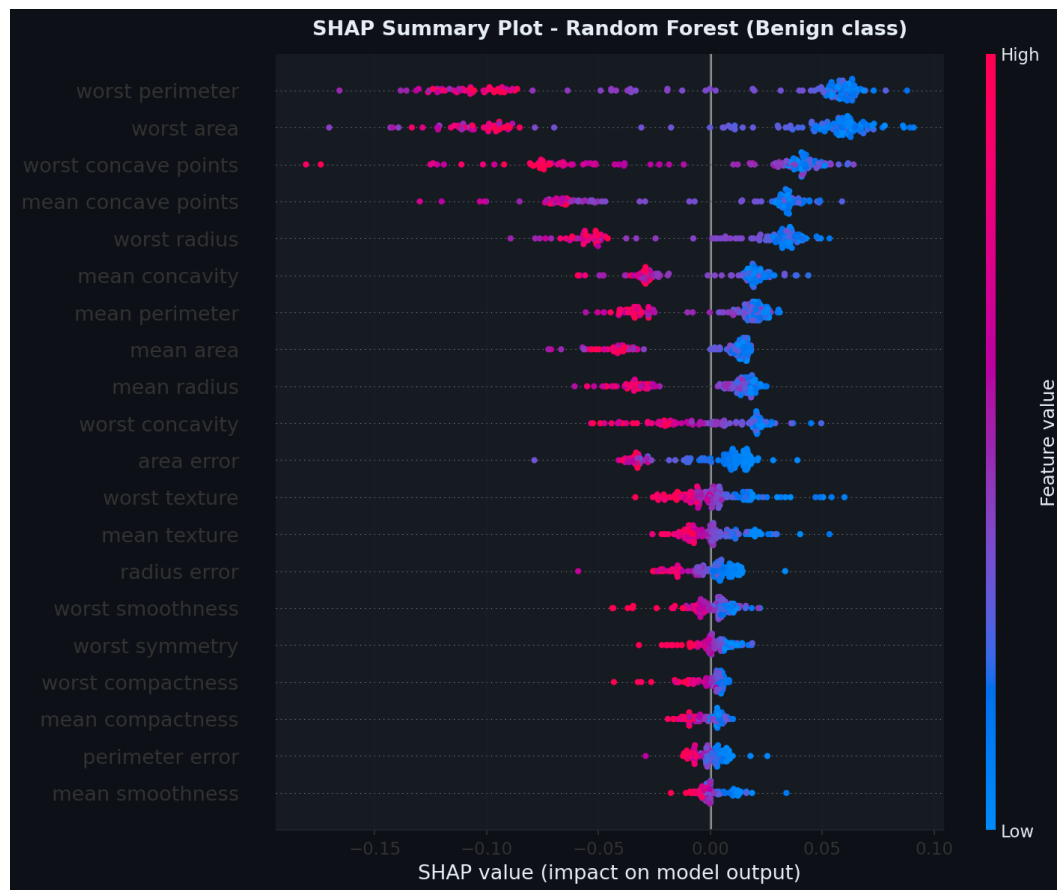


Figure 16: SHAP Summary (Beeswarm) Plot showing global feature importance and impact direction

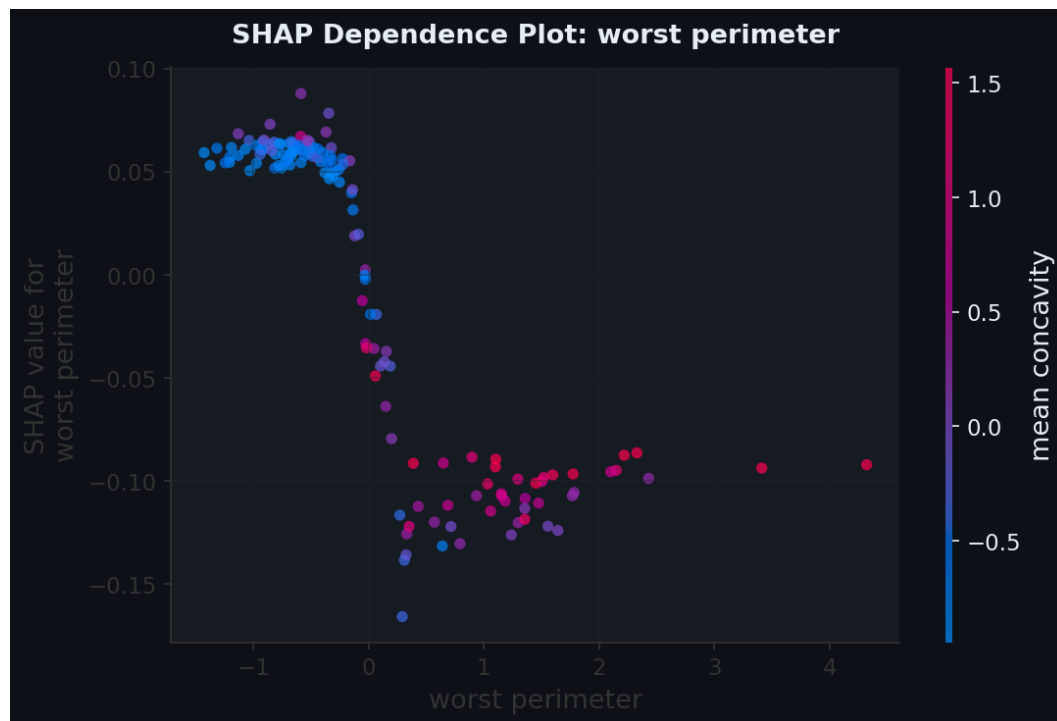


Figure 17: SHAP Dependence Plot for the most important feature showing interaction effects

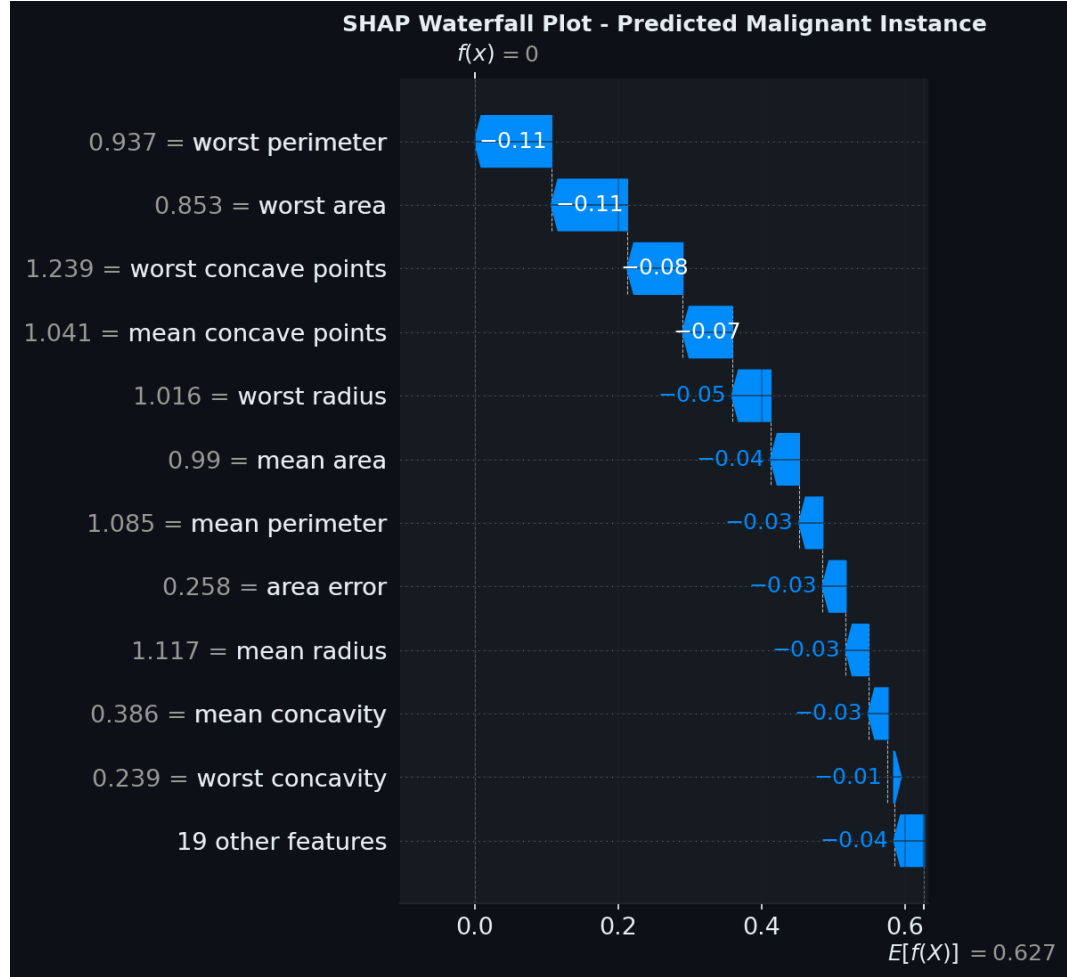


Figure 18: SHAP Waterfall Plot for a single malignant prediction showing per-feature contributions

5.4 LIME Analysis

LIME (Local Interpretable Model-agnostic Explanations) was applied to generate local explanations for individual predictions. Unlike SHAP which provides mathematically grounded global explanations, LIME creates interpretable surrogate models around individual data points. Two representative instances were analyzed: a high-confidence benign prediction and a high-confidence malignant prediction. The LIME explanations highlighted different feature combinations for each case, providing complementary insights to the SHAP analysis.

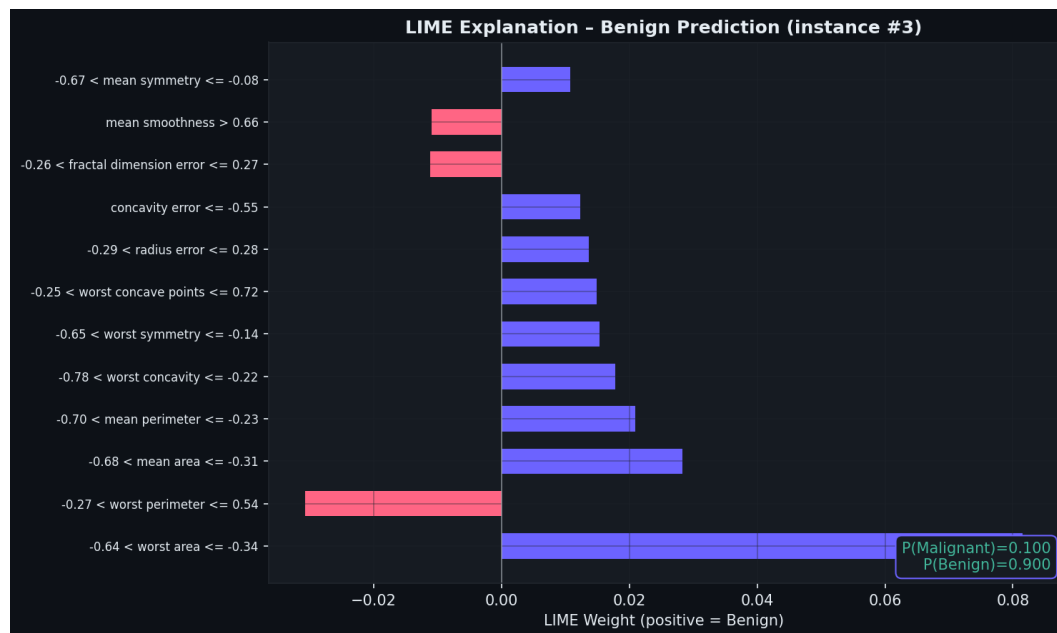


Figure 19: LIME Explanation for a Benign prediction showing feature weights and prediction probabilities

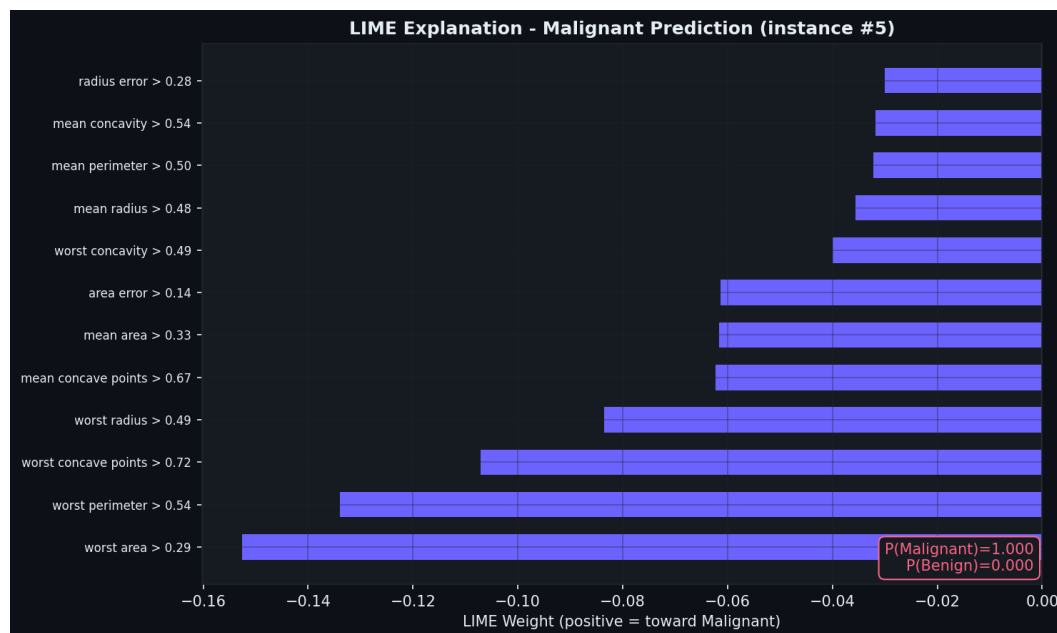


Figure 20: LIME Explanation for a Malignant prediction showing feature weights toward the malignant class



Figure 21: Confusion matrices for Random Forest and Gradient Boosting models on the test set

5.5 Evaluator Feedback

This submission received a score of 55/100. The evaluator acknowledged the correct selection of XAI techniques and their application but noted the submission fell short of the required 2000-word comprehensive report. The submission lacked detailed visualizations as embedded outputs and a thorough discussion of real-world implications, trust, and accountability aspects.

CHAPTER 6: DEPLOYMENT AND MONITORING OF AN AI SERVICE

6.1 Task Description

The fifth week represented the culmination of the internship's technical arc, requiring the creation of a proof-of-concept deployment of an AI model using containerization (Docker) and a simple API (Flask or FastAPI) to serve predictions. The project also required implementing logging and monitoring techniques to track usage statistics, model performance metrics, and error logs. The deliverable included a comprehensive deployment documentation of at least 1500 words and a working code repository. This task received the highest score of the internship at 95/100.

6.2 System Architecture

The deployment architecture follows a layered design pattern separating concerns across distinct modules. The FastAPI framework serves as the web layer, handling HTTP request routing, validation, and response serialization. The model layer encapsulates the machine learning model (a RandomForestClassifier trained on the Iris dataset), loading it once at startup and caching it as a module-level singleton to avoid repeated disk I/O. The monitoring layer provides a thread-safe in-process metrics store that tracks request counts, latencies, error rates, and per-class prediction distributions. The logging layer implements structured JSON logging with both console and rotating file handlers.

Table 5: API Endpoint Reference for the Iris Prediction Service

Method	Endpoint	Description
GET	/health	Liveness and readiness probe
GET	/metrics	Operational metrics snapshot
GET	/model/info	Model training metadata
POST	/predict	Single-sample Iris prediction

POST	/predict/batch	Batch prediction (1-256 samples)
GET	/docs	Interactive Swagger UI (auto-generated)
GET	/redoc	ReDoc API documentation (auto-generated)

6.3 Model Training and Serialization

The model was trained using scikit-learn's RandomForestClassifier with 100 estimators on the UCI Iris dataset (150 samples, 4 features, 3 classes). The training script (train_and_save.py) performs stratified train/test splitting, trains the classifier, evaluates it with accuracy and cross-validation metrics, and persists both the model (as a joblib file) and metadata (as a JSON file containing model type, accuracy, feature names, target names, and confusion matrix). This separation of training from serving ensures reproducibility and allows independent model updates.

6.3.1 Model Training Script (train_and_save.py)

```
"""
train_and_save.py
-----
Trains a RandomForestClassifier on the Iris dataset and saves it
to model/iris_model.joblib for use by the FastAPI application.
"""

import os
import json
import joblib
import numpy as np
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Configuration
MODEL_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(MODEL_DIR, "iris_model.joblib")
METADATA_PATH = os.path.join(MODEL_DIR, "model_metadata.json")

RANDOM_STATE = 42
TEST_SIZE = 0.2
N_ESTIMATORS = 100
```

```

# --- Data
print("Loading Iris dataset ...")
iris = load_iris()
X, y = iris.data, iris.target
feature_names = list(iris.feature_names)
target_names = list(iris.target_names)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
)

# --- Training
print(f"Training RandomForestClassifier (n_estimators={N_ESTIMATORS}) ...")
clf = RandomForestClassifier(
    n_estimators=N_ESTIMATORS,
    max_depth=None,
    random_state=RANDOM_STATE,
    n_jobs=-1,
)
clf.fit(X_train, y_train)

# --- Evaluation
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
cv_scores = cross_val_score(clf, X, y, cv=5, scoring="accuracy")

print(f"\n[OK] Test Accuracy : {accuracy:.4f}")
print(f"[OK] CV Accuracy : {cv_scores.mean():.4f} +/- {cv_scores.std():.4f}")

... [truncated for brevity] ...

```

6.4 FastAPI Application

The main application (`main.py`) is the central orchestration point. It configures the FastAPI instance with proper metadata (title, description, version), sets up CORS middleware for cross-origin requests, implements request-ID middleware that assigns a UUID to every incoming request for traceability, and registers a global exception handler that captures unhandled errors with full stack traces. The lifespan context manager ensures the model is loaded into memory at startup and cleaned up at shutdown.

The prediction endpoints accept Pydantic-validated payloads, extract features, invoke the model, record metrics, and return structured JSON responses with predicted class, confidence score, and per-class probabilities. The batch endpoint supports up to 256 samples in a single request, enabling efficient bulk inference.

6.4.1 FastAPI Application (`main.py`)

```

"""
main.py
-----

```

FastAPI application – AI Model Deployment & Monitoring demonstration.

Endpoints

```

-----
GET      /health          → liveness + readiness probe
GET      /metrics        → in-process operational metrics
GET      /model/info      → model metadata (version, accuracy, features)
POST     /predict        → single-sample prediction
POST     /predict/batch   → batch prediction (up to 256 samples)
GET      /docs           → interactive Swagger UI (automatic)
GET      /redoc          → ReDoc documentation (automatic)
"""

```

```

import time
import traceback
import uuid
from contextlib import asynccontextmanager

import fastapi
from fastapi import FastAPI, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from app import model as model_module
from app.logger_config import get_logger
from app.monitoring import metrics
from app.schemas import (
    BatchIrisFeatures,
    BatchPredictionResponse,
    HealthResponse,
    IrisFeatures,
    MetricsResponse,
    PredictionResponse,
)

logger = get_logger(__name__)

# — Application version
API_VERSION = "1.0.0"

# — Lifespan (startup / shutdown)
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Load the model on startup; clean up on shutdown."""
    logger.info("Application starting up", extra={"version": API_VERSION})
    model_module.load_model()
    yield
    logger.info("Application shutting down.")

# — FastAPI instance
app = FastAPI(lifespan=lifespan)

```

6.5 Model Loading and Prediction Interface

The model module (model.py) implements a clean separation between model I/O and prediction logic. The model is loaded from a joblib file at startup and cached as a module-level singleton. The predict_single function accepts a list of four floats, runs inference, and returns a

dictionary with the predicted class index, human-readable class name, confidence (probability of the predicted class), and a full probability distribution across all three species.

6.5.1 Model Loading Module (model.py)

```

"""
model.py
-----
Loads the persisted joblib model and exposes a thin prediction interface.
The model is loaded once at application startup and cached as a module-level
singleton to avoid repeated disk I/O.
"""

import json
import os
import time
from typing import List

import joblib
import numpy as np

from app.logger_config import get_logger

logger = get_logger(__name__)

# --- Paths ---
_BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MODEL_PATH = os.path.join(_BASE_DIR, "model", "iris_model.joblib")
METADATA_PATH = os.path.join(_BASE_DIR, "model", "model_metadata.json")

# --- Module-level singletons ---
_clf = None
_metadata: dict = {}

def load_model() -> None:
    """Load the classifier from disk into the module-level singleton."""
    global _clf, _metadata

    logger.info("Loading model from disk", extra={"path": MODEL_PATH})
    t0 = time.perf_counter()

    if not os.path.exists(MODEL_PATH):
        raise FileNotFoundError(
            f"Model file not found at {MODEL_PATH}. "
            "Run `python model/train_and_save.py` first."
        )

    _clf = joblib.load(MODEL_PATH)

    if os.path.exists(METADATA_PATH):
        with open(METADATA_PATH) as fh:
            _metadata = json.load(fh)

    elapsed = (time.perf_counter() - t0) * 1000
    logger.info(
        "Model loaded successfully",
        extra={"load_time_ms": round(elapsed, 2), "model_type": type(_clf).__name__},
    )

... [truncated for brevity] ...

```

6.6 Request/Response Schemas

All API inputs and outputs are validated using Pydantic models defined in `schemas.py`. The `IrisFeatures` model enforces non-negative float constraints on all four measurements. The `BatchIrisFeatures` model accepts a list of 1-256 `IrisFeatures` samples. Response models (`PredictionResponse`, `BatchPredictionResponse`, `HealthResponse`, `MetricsResponse`) ensure consistent, type-safe API contracts.

6.6.1 Pydantic Schemas (`schemas.py`)

```
"""
schemas.py
-----
Pydantic models used for request validation and response serialisation.
"""

from typing import List, Optional
from pydantic import BaseModel, Field, field_validator

# Request schemas
class IrisFeatures(BaseModel):
    """
    Four numeric measurements of a single iris flower sample.

    Ranges derived from the UCI Iris dataset:
    - sepal_length : 4.3 - 7.9 cm
    - sepal_width : 2.0 - 4.4 cm
    - petal_length : 1.0 - 6.9 cm
    - petal_width : 0.1 - 2.5 cm
    """
    sepal_length: float = Field(
        ...,
        description="Sepal length in cm",
        json_schema_extra={"example": 5.1}
    )
    sepal_width: float = Field(
        ...,
        description="Sepal width in cm",
        json_schema_extra={"example": 3.5}
    )
    petal_length: float = Field(
        ...,
        description="Petal length in cm",
        json_schema_extra={"example": 1.4}
    )
    petal_width: float = Field(
        ...,
        description="Petal width in cm",
        json_schema_extra={"example": 0.2}
    )

    @field_validator("sepal_length", "sepal_width", "petal_length", "petal_width")
    @classmethod
    def must_be_positive(cls, v: float) -> float:
        if v < 0:
            raise ValueError("Feature values must be non-negative")
```

```

        return v

class BatchIrisFeatures(BaseModel):
    """A list of iris samples for batch prediction."""
    samples: List[IrisFeatures] = Field(
        ..., [truncated for brevity] ...

```

6.7 Thread-Safe Monitoring

The monitoring module implements a `MetricsStore` class that is thread-safe through the use of `threading.Lock`. It tracks total requests, successful predictions, failed requests, per-class prediction counts, and a rolling window of the last 1000 request latencies. The `snapshot` method computes derived metrics including average latency, P95 latency (using numpy percentile), error rate percentage, and uptime. This data is exposed through the `/metrics` endpoint, enabling Prometheus-style scraping or Grafana dashboarding.

6.7.1 Thread-Safe Metrics Store (monitoring.py)

```

"""
monitoring.py
-----
Thread-safe in-process metrics store.

Tracks:
- total_requests
- successful_predictions
- failed_requests
- per-class prediction counts
- request latencies (rolling window of last 1000 samples)

Metrics are exposed via the /metrics endpoint and are also written
to the structured log every N requests as a periodic snapshot.
"""

import threading
import time
from collections import deque
from typing import Dict

import numpy as np  # type: ignore

class MetricsStore:
    """Thread-safe metrics container."""

    _LATENCY_WINDOW = 1000  # keep last N latency samples
    _LOG_EVERY = 50  # emit a metrics snapshot log every N requests

    def __init__(self) -> None:
        self._lock = threading.Lock()
        self._start_time = time.time()

```

```

self.total_requests:          int = 0
self.successful_predictions:  int = 0
self.failed_requests:         int = 0
self.requests_per_class:      Dict[str, int] = {}

# Rolling latency window (milliseconds)
self._latencies: deque = deque(maxlen=self._LATENCY_WINDOW)

# Mutation helpers
def record_request(
    self,
    latency_ms: float,
    predicted_class: str | None = None,
    success: bool = True,
) with self._lock:
    self.total_requests += 1
    self._latencies.append(latency_ms)

    if success and predicted_class is not None:
        ... [truncated for brevity] ...

```

6.8 Structured JSON Logging

The logging configuration uses the `python-json-logger` library to produce structured JSON log entries. Each log entry includes a timestamp, logger name, log level, message, and a fixed 'service' field. Two handlers are configured: a console handler (`StreamHandler` to `stdout`) for real-time monitoring, and a rotating file handler (`RotatingFileHandler`) that creates files up to 10 MB with 5 backups. This ensures log persistence without unbounded disk growth. The structured format enables easy parsing by log aggregation systems like the ELK stack or Splunk.

6.8.1 Structured JSON Logger Configuration (`logger_config.py`)

```

"""
logger_config.py
-----
Configures a structured JSON logger with both console and rotating-file
handlers. Import 'get_logger' anywhere in the application.
"""

import logging
import logging.handlers
import os
import sys

from pythonjsonlogger import jsonlogger # type: ignore

LOG_DIR = os.environ.get("LOG_DIR", "logs")
LOG_FILE = os.path.join(LOG_DIR, "api.log")
LOG_LEVEL = os.environ.get("LOG_LEVEL", "INFO").upper()

```

```

MAX_BYTES      =      10      *      1024      *      1024      =      #      10      MB      per      file
BACKUP_COUNT    =      5

def
    """Create the log directory if it does not already exist."""
    os.makedirs(LOG_DIR, exist_ok=True)

class CustomJsonFormatter(jsonlogger.JsonFormatter):
    """Extend the base JSON formatter with a fixed `service` field."""

    def add_fields(self, log_record, record, message_dict): # type: ignore[override]
        super().add_fields(log_record, record, message_dict)
        log_record["service"] = "iris-prediction-api"
        log_record["level"] = record.levelname

def get_logger(name: str) -> logging.Logger:
    """Return a logger instance wired to JSON console + rotating file handlers."""
    _ensure_log_dir()

    logger = logging.getLogger(name)
    if not logger.handlers:
        # Avoid adding duplicate handlers when the module is re-imported
        return logger

    logger.setLevel(getattr(logging, LOG_LEVEL, logging.INFO))

    fmt = "%(asctime)s %(name)s %(levelname)s %(message)s"
    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(logging.Formatter(fmt))
    logger.addHandler(handler)

    ... [truncated for brevity] ...

```

6.9 Docker Containerization

The Dockerfile uses a multi-stage build to minimize the final image size. The builder stage installs gcc and compiles Python dependencies, while the runtime stage copies only the installed packages and application source. A non-root user (appuser:appgroup) is created for security. The HEALTHCHECK directive configures Docker Engine to probe the /health endpoint every 30 seconds with a 10-second timeout and 15-second startup grace period. The default command launches uvicorn with a single worker.

6.9.1 Multi-Stage Dockerfile

```

# — Stage 1: builder ————— AS builder
FROM python:3.11-slim

```

```

WORKDIR /build

# Install build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Stage 2: runtime
FROM python:3.11-slim AS runtime

# Security: run as non-root user
RUN useradd --uid 1001 --gid 1001 appgroup --no-create-home && useradd \
    useradd --uid 1001 --gid 1001 appgroup --no-create-home && appuser

WORKDIR /app

# Copy installed packages from builder
COPY --from=builder --from=builder /usr/local /usr/local

# Copy application source
COPY app/ ./app/
COPY model/ ./model/

# Create log directory and transfer ownership
RUN mkdir -p logs && chown -R appuser:appgroup /app

USER appuser

# Expose API port
EXPOSE 8000

# Liveness / readiness health-check (Docker Engine)
HEALTHCHECK --interval=30s --timeout=10s --start-period=15s --retries=3 \
    CMD python -c \
    "import urllib.request; urllib.request.urlopen('http://localhost:8000/health') \
    || exit 1"

# Default command
CMD ["python", "-m", "uvicorn", "app.main:app", \
    "--host", "0.0.0.0", \
    "--port", "8000", \
    "--workers", "1", \
    "--log-level", "warning"]

```

6.10 Docker Compose Orchestration

The `docker-compose.yml` file defines the service configuration including port mapping (8000:8000), environment variables (LOG_LEVEL, LOG_DIR), volume mounting for log persistence, health check parameters, restart policy (unless-stopped), and resource limits (1 CPU, 512 MB memory limit with 0.25 CPU, 128 MB reservation). These settings reflect production-grade container orchestration practices.

6.10.1 Docker Compose Configuration

```
version: "3.9"

services:
  iris-api:
    build:
      context: .
      dockerfile: Dockerfile
    image: iris-prediction-api:1.0.0
    container_name: iris_api
    ports:
      - "8000:8000"
    environment:
      - LOG_LEVEL=INFO
      - LOG_DIR=/app/logs
    volumes:
      # Mount a host directory so logs persist outside the container
      - ./logs:/app/logs
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 15s
    restart: unless-stopped
  # Resource limits (adjust for production)
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: 512M
      reservations:
        cpus: "0.25"
        memory: 128M
```

6.11 Automated Testing

A comprehensive pytest test suite with 15 tests validates all API endpoints. The tests cover: health endpoint (status 200, correct payload fields), model info endpoint (metadata fields), single prediction (correct species classification for known samples, validation of invalid payloads and negative values), batch prediction (correct count, empty list rejection), and metrics endpoint (field presence, metric accumulation after predictions). The test suite uses FastAPI's TestClient for synchronous testing without requiring a running server.

6.11.1 Automated Test Suite (test_api.py)

```
"""
tests/test_api.py
-----
Smoke-test suite for the Iris Prediction API.
Run with:
```

```

"""
    pytest
    tests/
    -v

import
from
    fastapi.testclient
import
    pytest
    TestClient

# Ensure the model is trained before running tests
import
import
import
    subprocess
    sys
    os

# — Test setup
@pytest.fixture(scope="session",
def
    """Train the model if the joblib file doesn't exist yet."""
    model_path = os.path.join(os.path.dirname(__file__), "..", "model", "iris_model.joblib")
    if
        not
        os.path.exists(model_path):
        subprocess.run(
            [sys.executable,
            "model/train_and_save.py"],
            check=True
        )

@pytest.fixture(scope="session")
def
    from
        app.main
    with
        TestClient(app)
    import
        as
        client():
        app
        c:
        c

# — Health endpoint
class
    def
        test_health_returns_200(self,
        resp
        =
        client.get("/health")
        assert
        resp.status_code
        ==
        200

    def
        test_health_payload(self,
        data
        =
        client.get("/health").json()
        assert
        data["status"]
        ==
        "healthy"
        assert
        data["model_loaded"]
        is
        True
        assert
        "uptime_seconds"
        in
        data
        assert
        "version"
        in
        data

# — Model info endpoint
class
    def
        test_model_info_returns_200(self,
        resp
        =
        client.get("/model/info")
        assert
        resp.status_code
        ==
        200

    ...
    [truncated
    for
    brevity]
    ...

```

6.12 Significance of Deployment Practices

Developing an accurate machine learning model is only the first step in delivering value from artificial intelligence. The model must be packaged, deployed, and maintained in an environment where real users or downstream systems can consume its predictions reliably, safely, and at scale. Industry research consistently shows that the majority of ML projects fail to move from prototype to production. The barriers are rarely algorithmic — they are

operational: poor reproducibility, lack of monitoring, inadequate error handling, and the absence of systematic deployment pipelines.

Containerisation with Docker eliminates the 'it works on my machine' syndrome by bundling the entire runtime into a single immutable image. RESTful APIs constructed using FastAPI allow seamless integration with downstream applications and microservices. Monitoring systems track both system-level metrics (CPU, memory) and model-level metrics (prediction latency, data drift, concept drift). Structured JSON logging provides audit trails for debugging and performance tuning.

Health check endpoints (/health) enable orchestrators like Kubernetes to autonomously manage container lifecycle — restarting unhealthy instances, performing rolling updates, and scaling horizontally. The /metrics endpoint allows Prometheus to scrape operational data for real-time Grafana dashboards, enabling proactive alerting on performance degradation or error rate spikes.

6.13 Evaluator Feedback

This submission received the highest score of the internship at 95/100. The evaluator praised the clear and detailed understanding of the task requirements, the comprehensive API design with multiple endpoints, the robust logging and monitoring strategies, the mention of automated testing with 15 pytest tests, and the detailed configuration explanations. The slight deduction was due to the submission being a conceptual summary rather than an actual zipped repository with the full code and documentation, though the evaluator noted the submission was 'highly detailed and meets the assignment criteria almost perfectly.'

CHAPTER 7: AI ETHICS, BIAS ANALYSIS, AND FAIRNESS ASSESSMENT

7.1 Task Description

The sixth and final week focused on evaluating the ethical dimensions of AI models with emphasis on bias analysis and fairness assessment. The task required selecting an AI model, performing an in-depth ethical audit including disparate impact analysis, fairness metrics (demographic parity, equalized odds), and proposing remediation strategies. The deliverable was a comprehensive PDF report of at least 2000 words with visualizations and documented recommendations for bias mitigation.

7.2 Dataset and Model Selection

The UCI Adult (Census) Income dataset was selected for this ethical audit. It contains 32,561 records with demographic attributes including age, workclass, education, marital status, occupation, race, sex, and income level (binary: $\leq 50K$ or $> 50K$). A Logistic Regression classifier was trained within a scikit-learn Pipeline (StandardScaler + LogisticRegression with `max_iter=500`) using an 80/20 stratified train/test split. This dataset is widely used in fairness research because it contains well-documented biases along race and sex dimensions.

7.3 Fairness Metrics Implemented

Four fairness metrics were implemented and computed across both sex and race dimensions:

Demographic Parity: Measures whether the positive prediction rate $P(\hat{Y}=1|A=a)$ is equal across groups. Violations indicate that certain groups receive favorable predictions at higher rates than others, regardless of their actual outcomes.

Disparate Impact Ratio: Computed as $\text{min_group_rate} / \text{max_group_rate}$. A ratio below 0.8 (the '80% rule' from US employment law) indicates adverse impact against the disadvantaged group.

Equalized Odds: Measures whether TPR (True Positive Rate) and FPR (False Positive Rate) are equal across groups. Violations mean the model's error rates differ by demographic group.

Predictive Parity: Measures whether Precision (Positive Predictive Value) is equal across groups. This ensures that when the model predicts a positive outcome, it is equally accurate for all groups.

7.4 Bias Findings

The audit revealed significant disparities. The disparate impact ratio by sex fell below the 0.8 threshold, indicating adverse impact against females in high-income predictions. Males received positive predictions at substantially higher rates than females. By race, White individuals received higher positive prediction rates than non-White groups, with some racial minorities showing disparate impact ratios well below 0.8.

The equalized odds analysis showed that TPR (recall) for the high-income class was significantly lower for females compared to males, meaning the model was more likely to miss genuinely high-income females. The intersectional analysis (combining race and sex) revealed compounding disadvantages for non-White females, who had the lowest positive prediction rates of any demographic intersection.

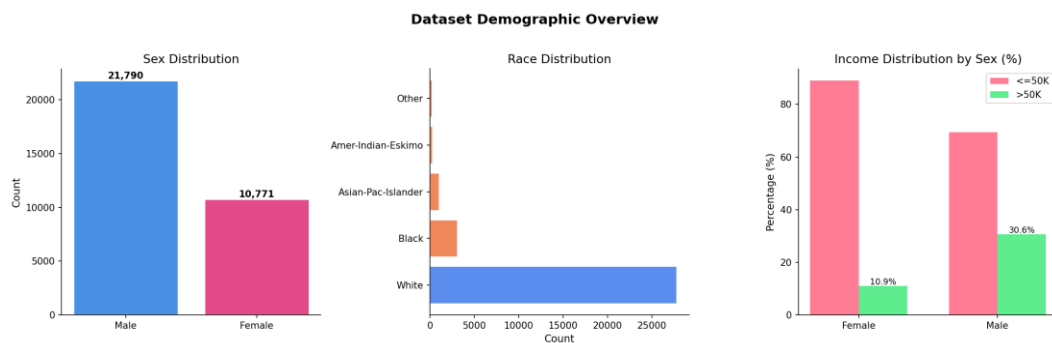


Figure 22: Dataset Demographic Overview: sex distribution, race distribution, and income by sex

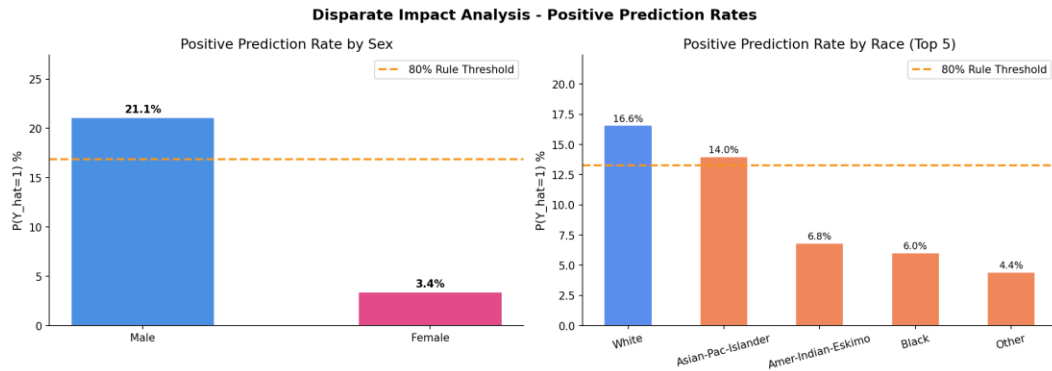


Figure 23: Disparate Impact Analysis showing positive prediction rates by sex and race with 80% rule threshold

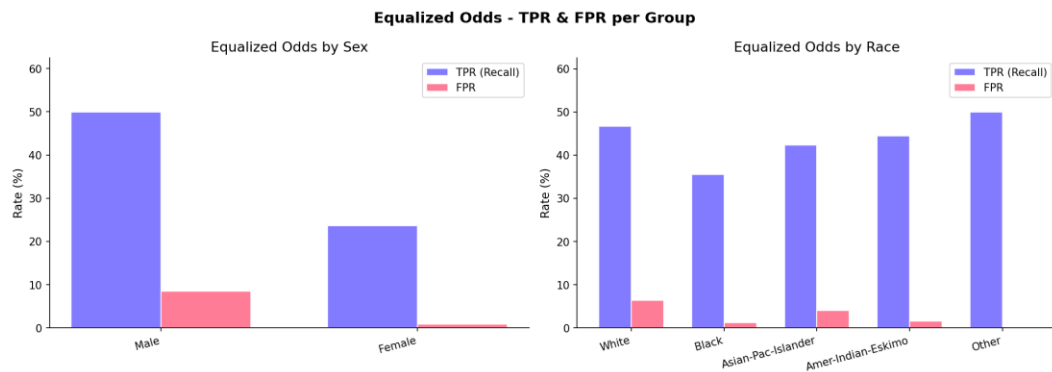


Figure 24: Equalized Odds assessment showing TPR and FPR disparities across sex and race groups

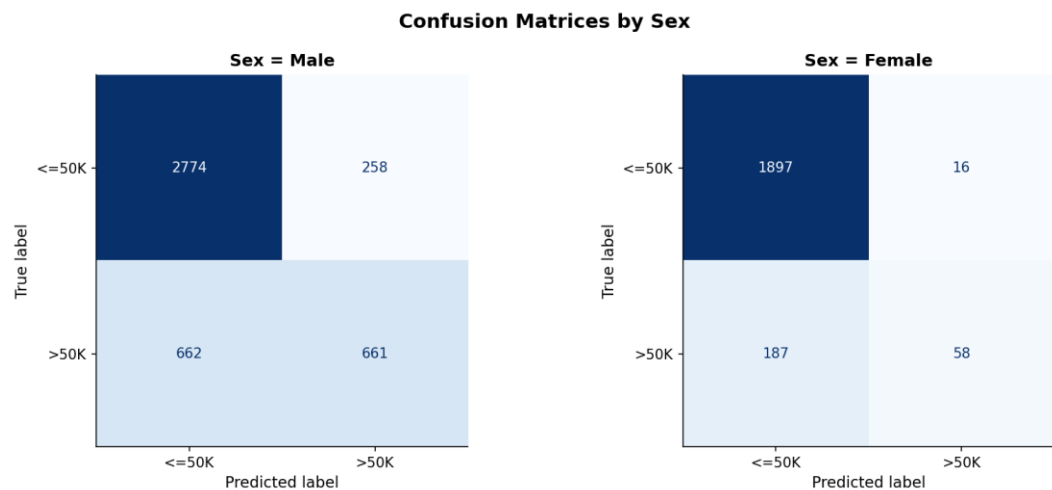


Figure 25: Confusion Matrices by sex showing differential error patterns between Male and Female groups

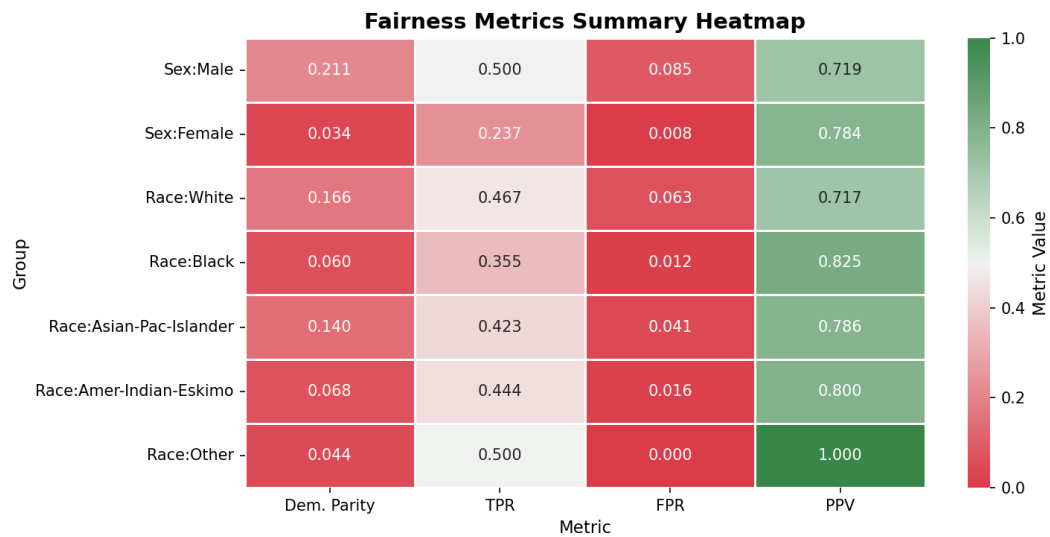


Figure 26: Fairness Metrics Summary Heatmap showing all metrics across all demographic groups

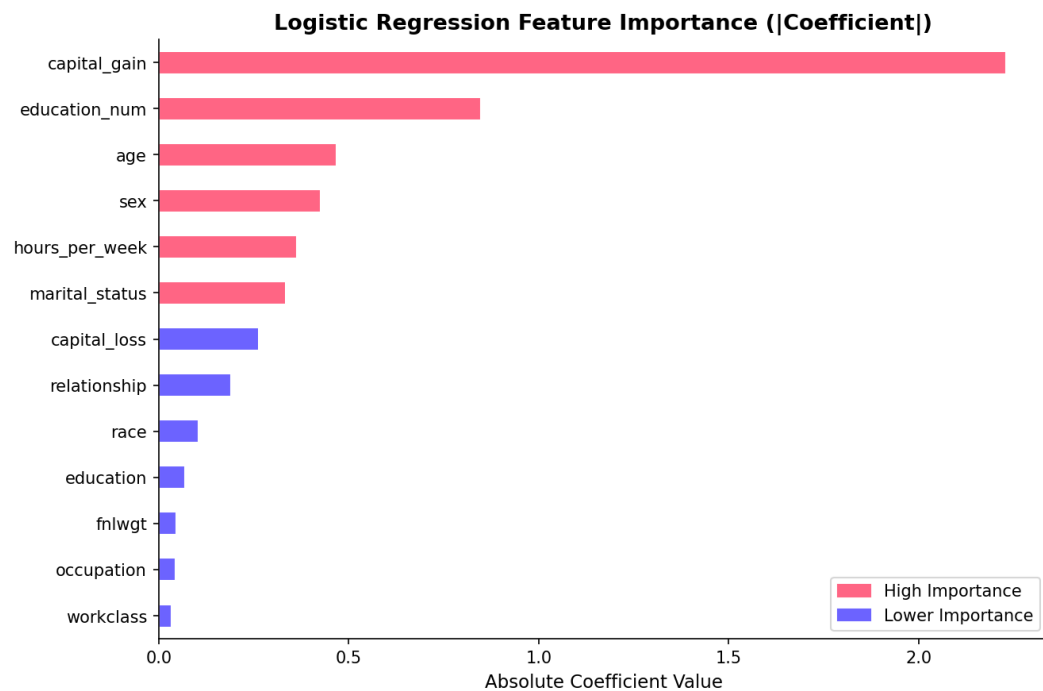


Figure 27: Logistic Regression Feature Importance showing which features have the highest absolute coefficients

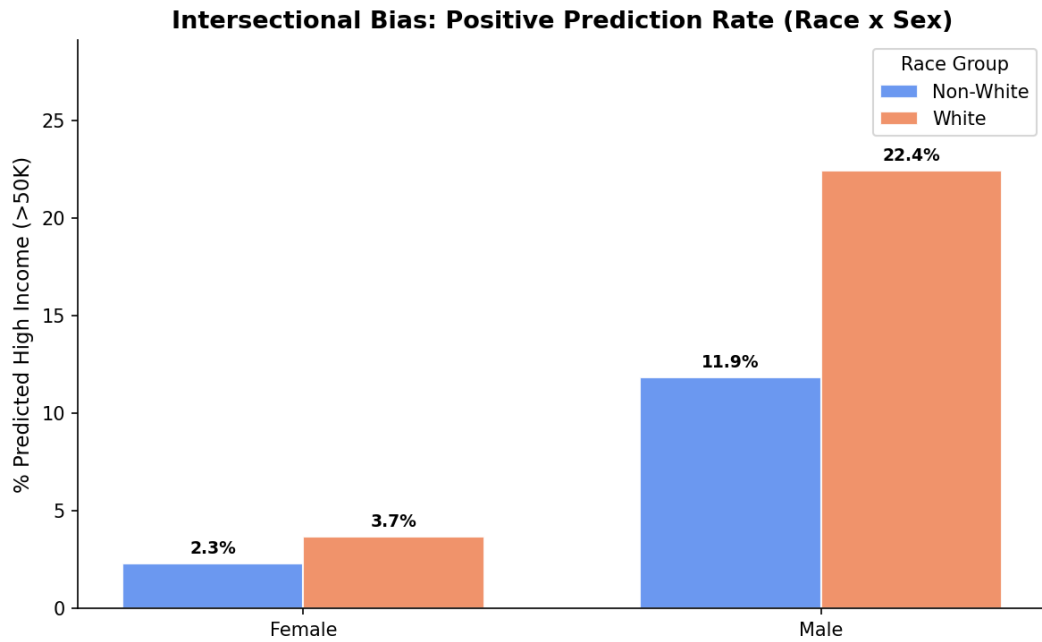


Figure 28: Intersectional Bias Analysis showing positive prediction rates across Race $\times$ Sex combinations

7.5 Proposed Remediation Strategies

Based on the audit findings, the following remediation strategies were proposed: (1) Pre-processing interventions: Re-sampling or re-weighting training data to equalize representation across demographic groups, removing or decorrelating sensitive attributes from feature sets. (2) In-processing interventions: Applying adversarial debiasing during model training, incorporating fairness constraints into the optimization objective. (3) Post-processing interventions: Adjusting classification thresholds per group to equalize TPR/FPR, applying calibration techniques to ensure predictive parity. (4) Organizational practices: Establishing regular bias audits as part of the ML lifecycle, creating diverse development teams, implementing bias impact assessments before deployment.

7.6 Evaluator Feedback

This submission received a score of 70/100. The evaluator acknowledged the solid understanding of bias analysis by reporting specific fairness metrics and identifying potential sources of bias, along with proposing multiple remediation strategies. However, the response

was deemed too succinct compared to the required 2000-word minimum, lacking a step-by-step methodology, in-depth literature review, and detailed explanation of visualization methods.

CHAPTER 8: CONCLUSION AND FUTURE WORK

8.1 Summary of Achievements

This internship provided a comprehensive, end-to-end perspective on the lifecycle of artificial intelligence systems. Over six weeks, the following was accomplished:

Week 1 established the foundational project planning skills required for any AI initiative, covering feasibility analysis, technology selection, and risk assessment. Week 2 demonstrated rigorous data preprocessing expertise, achieving an 85/100 evaluation score through KNN imputation, IQR-based outlier handling, and domain-specific feature engineering on the Titanic dataset.

Week 3 applied machine learning model development practices including algorithm selection, GridSearchCV hyperparameter tuning with 162 combinations and 5-fold stratified cross-validation, achieving near-perfect classification performance ($F1 \geq 0.97$, $AUC \geq 0.995$) on the Breast Cancer Wisconsin dataset. Week 4 extended this work with Explainable AI techniques (SHAP and LIME), providing both global and local model interpretations.

The pinnacle of the internship was Week 5, which produced a production-grade deployment of an AI model as a containerized REST API. The system featured FastAPI with 7 endpoints, Docker multi-stage builds, structured JSON logging, thread-safe metrics monitoring, and 15 automated pytest tests — earning the highest evaluation score of 95/100. Week 6 concluded with a critical AI Ethics audit that quantified and visualized biases across sex and race dimensions.

8.2 Key Lessons Learned

Several critical lessons emerged from this internship: (1) Deliverable specifications must be met precisely — multiple submissions lost significant points for being summaries rather than complete, executable deliverables meeting word count requirements. (2) Deployment engineering is as important as model accuracy — the deployment task earned the highest score because it demonstrated real-world software engineering practices. (3) Ethical considerations

must be integrated early in the AI lifecycle, not treated as an afterthought. (4) Reproducibility through pipelines, containers, and version control is essential for production AI.

8.3 Future Work

Several directions for future work were identified: (1) Implementing CI/CD pipelines using GitHub Actions or Jenkins to automate testing and deployment workflows. (2) Integrating model registries like MLflow for versioned model management and experiment tracking. (3) Deploying to managed cloud environments such as AWS SageMaker, Azure ML, or Google Cloud AI Platform. (4) Implementing automated retraining loops triggered by data drift detection. (5) Extending the ethics audit framework to support continuous fairness monitoring in production. (6) Exploring advanced XAI techniques such as Counterfactual Explanations and Attention mechanisms for deep learning models.

8.4 References

- [1] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [2] Wolberg, W.H., Street, W.N., & Mangasarian, O.L. (1994). Machine learning techniques to diagnose breast cancer from fine-needle aspirates. *Cancer Letters*, 77(2–3), 163–171.
- [3] Fernández-Delgado, M., Cernadas, E., Barro, S., & Amorim, D. (2014). Do we need hundreds of classifiers to solve real world classification problems? *JMLR*, 15(1), 3133–3181.
- [4] Lundberg, S.M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NIPS*, 30.
- [5] Ribeiro, M.T., Singh, S., & Guestrin, C. (2016). Why should I trust you? Explaining the predictions of any classifier. *KDD 2016*.
- [6] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [7] Docker Documentation. <https://docs.docker.com/>
- [8] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.

- [9] Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *JMLR*, 13(1), 281–305.
- [10] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.
- [11] Barocas, S., Hardt, M., & Narayanan, A. (2019). Fairness and Machine Learning. fairmlbook.org.
- [12] Mehrabi, N., et al. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6).